DiaTwin - Complete Application Structure

Unified Architecture and File Organization

Document Version: 3.0
Last Updated: February 11, 2026
Purpose: Complete structural reference for the entire DiaTwin application

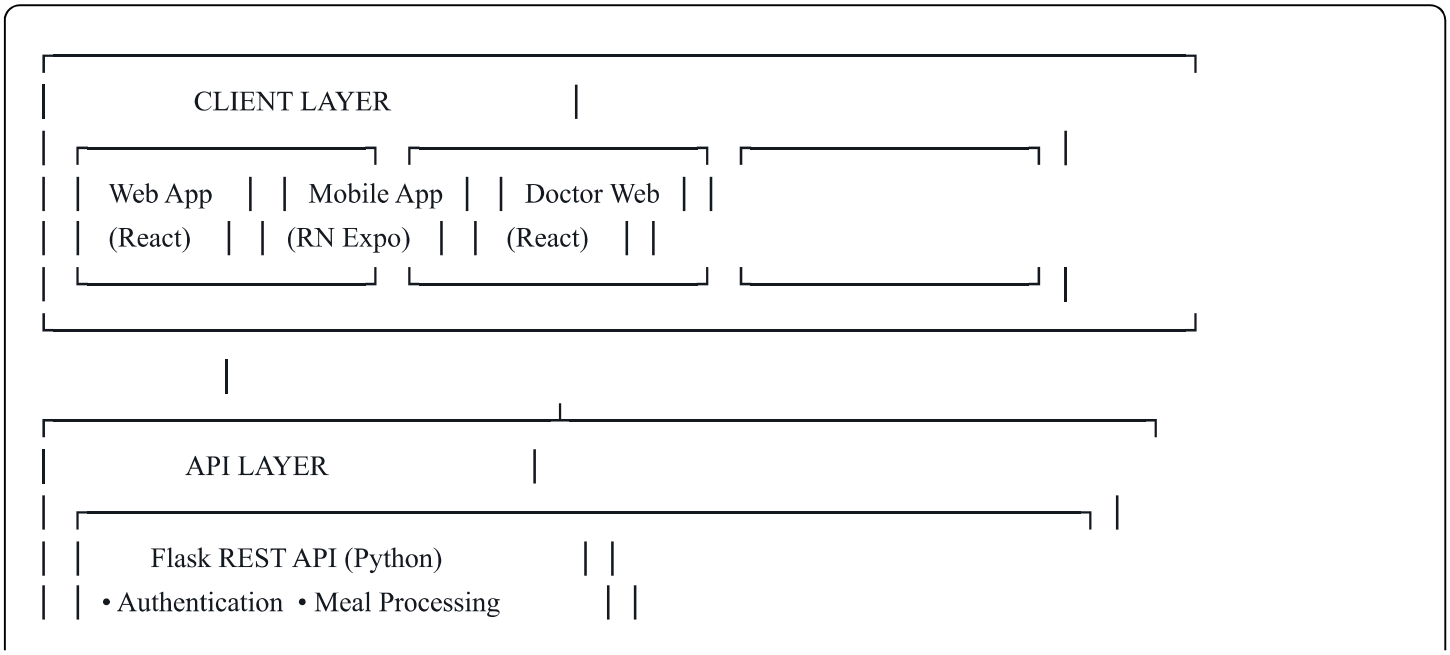
---

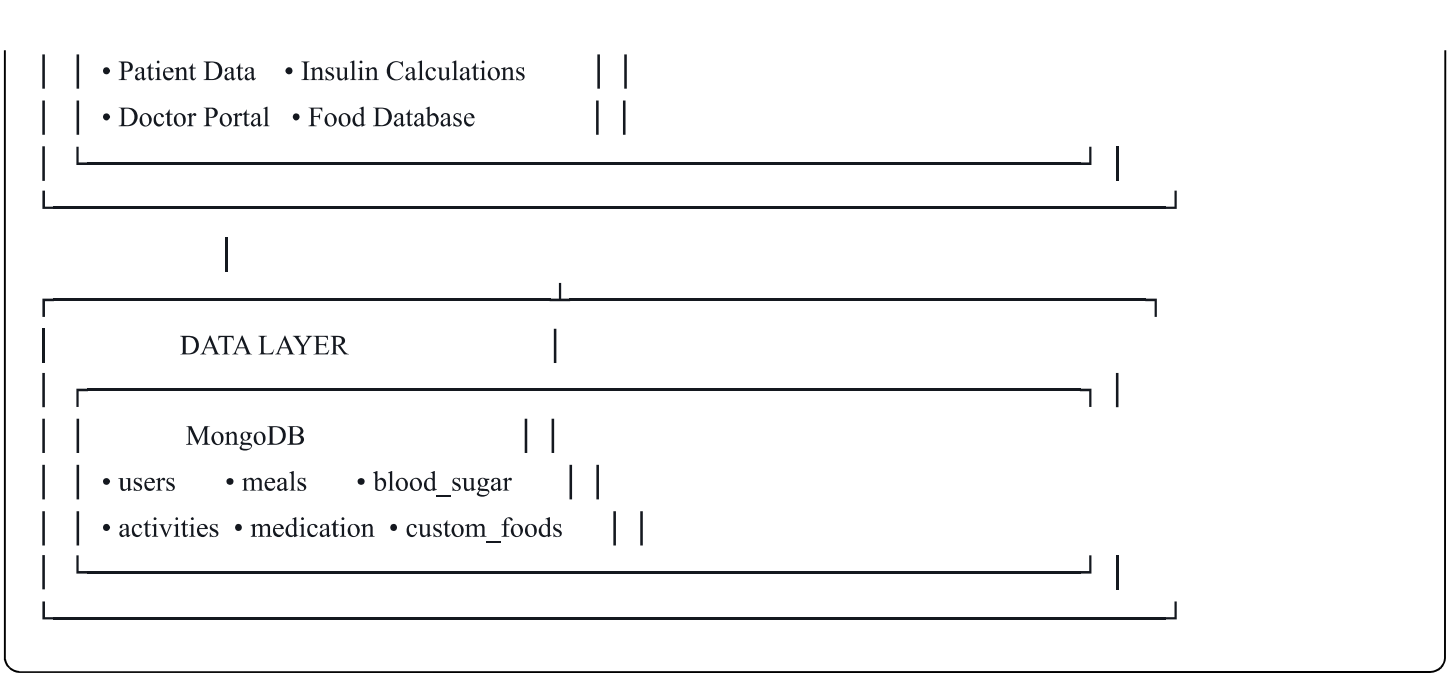
📋 TABLE OF CONTENTS

1. Project Overview
2. Directory Structure
3. Backend Structure
4. Frontend Structure
5. Mobile Structure
6. Data Flow Architecture
7. Dependency Map
8. API Structure
9. Component Hierarchy
10. File Reference Guide

🎯 PROJECT OVERVIEW

Application Layers





📁 DIRECTORY STRUCTURE

Complete Project Tree

```
native3/
├── .expo/                      # Expo configuration
│   ├── devices.json
│   └── README.md
│
├── backend/                    # Python Flask API
│   ├── models/                 # Data models
│   │   ├── __init__.py
│   │   └── food_data.py        # 500+ food items, 14 categories
│   │
│   ├── routes/                 # API endpoints (Blueprints)
│   │   ├── __init__.py
│   │   ├── auth_routes.py      # Login, register, user management
│   │   ├── doctor_routes.py    # Doctor portal, patient management
│   │   ├── food_routes.py      # Food database, search
│   │   ├── import_routes.py    # CSV/JSON import
│   │   ├── meal_routes.py      # Meal history, MOB tracking
│   │   ├── medication_routes.py  # Insulin logs, schedules, IOB
│   │   ├── patient_routes.py   # Patient-specific endpoints
│   │   └── test_routes.py      # Testing/debugging endpoints
│   │
│   ├── services/               # Business logic layer
│   │   └── food_service.py     # Food search, lookup utilities
│   │
│   ├── tests/                  # Test suite
│   │   ├── __init__.py
```

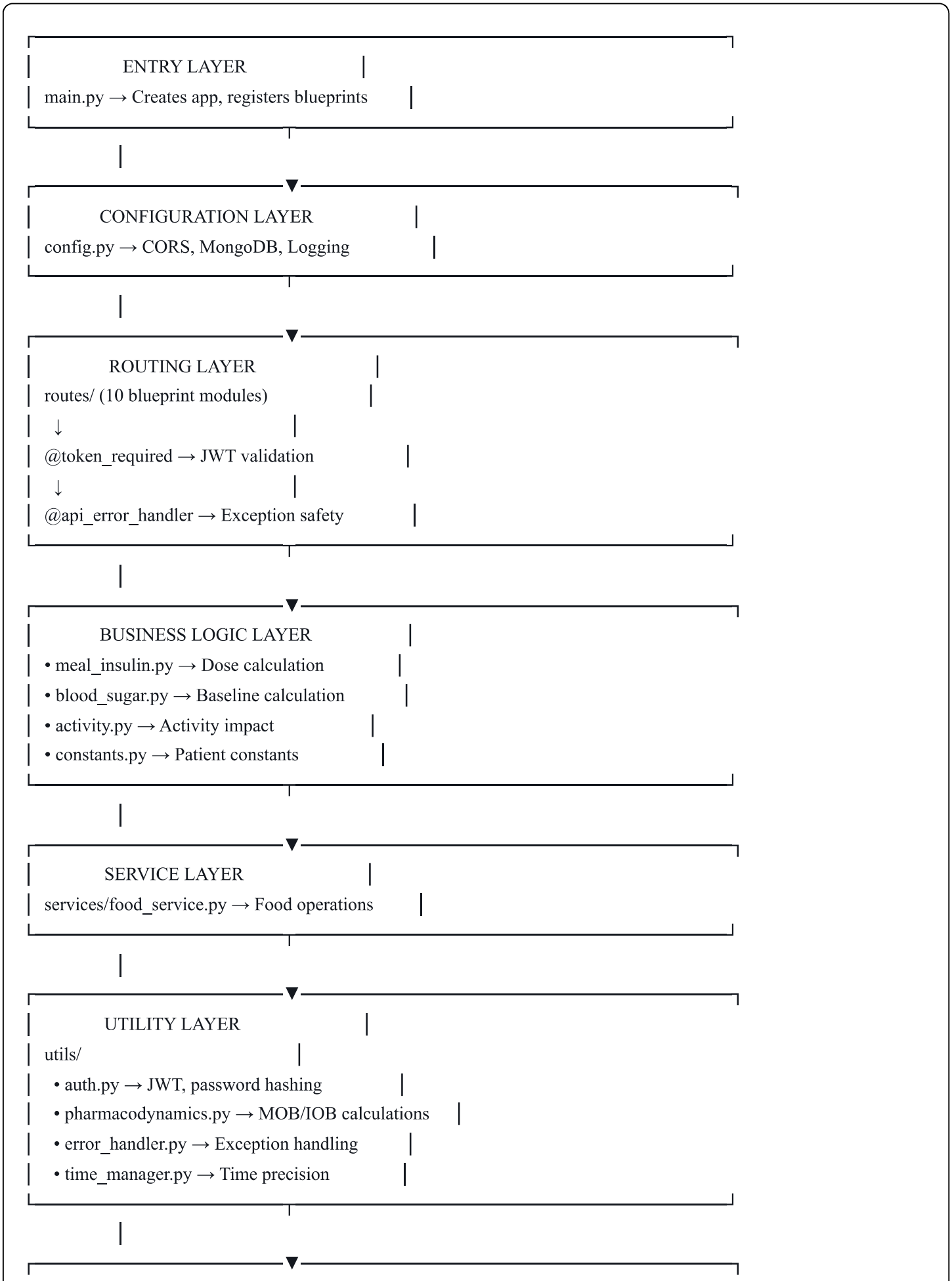
```
│   │   └── test_mob_iob_integration.py  # Frontend-backend sync tests
│   │
│   ├── utils/                    # Utility modules
│   │   ├── __init__.py
│   │   ├── auth.py               # JWT decorators, password hashing
│   │   ├── error_handler.py      # Exception handling decorator
│   │   └── pharmacodynamics.py   # MOB/IOB gamma calculations
│   │
│   ├── __init__.py               # App factory
│   ├── activity.py               # Activity tracking blueprint
│   ├── API_CHANGES.md            # API changelog
│   ├── blood_sugar.py            # Blood glucose blueprint
│   ├── code doc.txt              # Legacy documentation
│   ├── config.py                 # Flask, CORS, MongoDB config
│   ├── constants.py              # Patient constants, pharma profiles
│   ├── export_constants.py       # Frontend export utility
│   ├── main.py                   # Entry point (runs on port 5000)
│   ├── meal_insulin.py           # Meal submission, dose calculation
│   ├── requirements.txt          # Python dependencies
│   ├── time_manager.py           # Time precision utilities
│   └── tree_gen.py               # Directory structure generator
│
├── draft/                        # Development artifacts
│   ├── Artifacts/                # Experimental components
│   │   ├── bg_system_comparison.tsx
│   │   ├── BGEffectPatternVisualizer.js (v1-v3)
│   │   ├── EnhancedIOBModel.js (v1-v3)
│   │   ├── mob_iob.js (v1-v2)
│   │   └── t1d-model-explanation.tsx
│   │
│   ├── New folder/               # Legacy components
│   │   ├── activityeffectdisplay.txt
│   │   ├── BG_Effect.js
│   │   └── BloodSugarDataContext.js
│   │
│   └── [Various old implementations]
│
├── frontend/                     # React Web Application
│   ├── public/                   # Static assets
│   │   ├── favicon.ico
│   │   ├── index.html
│   │   ├── logo192.png
│   │   ├── logo512.png
│   │   ├── manifest.json
│   │   └── robots.txt
│   │
│   ├── src/
```

```
│   │   ├── components/           # React components (33 files)
│   │   │   ├── Charts/           # CSS stylesheets
│   │   │   │   ├── dashboard-components.css
│   │   │   │   ├── effects-visualization.css
│   │   │   │   ├── global-visualization.css
│   │   │   │   ├── input-components.css
│   │   │   │   ├── new-global-viz.css
│   │   │   │   ├── portal-styling.css
│   │   │   │   ├── SMBGMonitoringDashboard.css
│   │   │   │   └── Z Mapping.txt
│   │   │   │
│   │   │   ├── ActiveEffectsDisplay.js         # 1,958 lines
│   │   │   ├── ActivityRecording.js
│   │   │   ├── ActivityVisualization.js
│   │   │   ├── AdminDashboard.js
│   │   │   ├── BloodSugarChart.js
│   │   │   ├── BloodSugarInput.js
│   │   │   ├── BloodSugarVisualization.js
│   │   │   ├── CGMMonitoringDashboard.js
│   │   │   ├── ClinicalValidationDashboard.js
│   │   │   ├── CombinedGlucoseInsulinChart.js
│   │   │   ├── DoctorDashboard.js
│   │   │   ├── DoctorManagement.js
│   │   │   ├── DoctorMealHistory.js
│   │   │   ├── EffectsVisualization.js
│   │   │   ├── EnhancedPatientConstants.js
│   │   │   ├── EnhancedPatientConstantsCalc.js
│   │   │   ├── EnhancedPatientConstantsUI.js
│   │   │   ├── FoodDatabase.js
│   │   │   ├── FoodSection.js
│   │   │   ├── GlycemicResponseTracker.js
│   │   │   ├── ImportExport.js
│   │   │   ├── InsulinInput.js
│   │   │   ├── InsulinVisualization.js
│   │   │   ├── MealHistory.js
│   │   │   ├── MealInput.js
│   │   │   ├── MealVisualizationmealsonly.js
│   │   │   ├── MedicationSchedule.js
│   │   │   ├── NetEffectDisplay.js
│   │   │   ├── PatientConstants.js
│   │   │   ├── PatientDashboard.js
│   │   │   ├── Register.js
│   │   │   ├── SignIn.js
│   │   │   ├── SMBGMonitoringDashboard.js
│   │   │   ├── Test.js
│   │   │   └── TimeInput.js
│   │   │
```

```
│   │   ├── constants/              # Configuration
│   │   │   ├── index.js
│   │   │   └── shared_constants.js  # AUTO-GENERATED from backend
│   │   │
│   │   ├── contexts/               # React Context Providers
│   │   │   ├── BloodSugarDataContext.js  # 2,279 lines - CRITICAL
│   │   │   ├── ConstantsContext.js
│   │   │   └── TimeContext.js
│   │   │
│   │   ├── hooks/                  # Custom React hooks
│   │   │   └── useVisualizationData.js
│   │   │
│   │   ├── utils/                  # Utility services (14 files)
│   │   │   ├── bloodGlucoseEstimationService.js   # 1,681 lines
│   │   │   ├── ChartUtils.js                      # 1,377 lines
│   │   │   ├── confidenceVisualization.js
│   │   │   ├── DataImportHelper.js
│   │   │   ├── insulinCalculationService.js
│   │   │   ├── insulinPharmacodynamics.js
│   │   │   ├── insulinStackingAnalyzer.js
│   │   │   ├── mealCalculationService.js
│   │   │   ├── mealPharmacodynamics.js
│   │   │   ├── mealStackingAnalyzer.js
│   │   │   ├── netEffectCalculationService.js     # 1,695 lines
│   │   │   ├── readme.txt
│   │   │   ├── SMBGAnalysisService.js
│   │   │   ├── TimeManager.js
│   │   │   └── validationMetrics.js
│   │   │
│   │   ├── App.css
│   │   ├── App.js                  # Root component with context providers
│   │   ├── App.test.js
│   │   ├── index.css
│   │   ├── index.js                # Entry point
│   │   ├── logo.svg
│   │   ├── reportWebVitals.js
│   │   └── setupTests.js
│   │
│   ├── .babelrc
│   ├── babel.config.js
│   ├── package.json
│   └── README.md
│
└── [Root configuration files]
```

🔧 BACKEND STRUCTURE

Layer Architecture



```
┌─────────────────────────────────────────┐
│              DATA LAYER                 │
│  models/food_data.py → Food database    │
│  MongoDB → Document storage             │
└─────────────────────────────────────────┘
```

Blueprint Organization

```python
# main.py blueprint registration order

blueprints = [
    # Core functionality first
    (food_routes, ''),        # Food database
    (auth_routes, ''),        # Authentication
    (doctor_routes, ''),      # Doctor portal
    (patient_routes, ''),     # Patient management
    (test_routes, ''),        # Testing endpoints

    # Domain modules
    (meal_insulin_bp, ''),    # Meal processing
    (activity_bp, ''),        # Activity tracking
    (blood_sugar_bp, ''),     # Blood glucose

    # Support modules
    (medication_routes, ''),  # Medication tracking
    (import_routes, ''),      # Data import
    (meal_bp, ''),            # Meal history (last to avoid conflicts)
]
```

Database Collections

```javascript
```

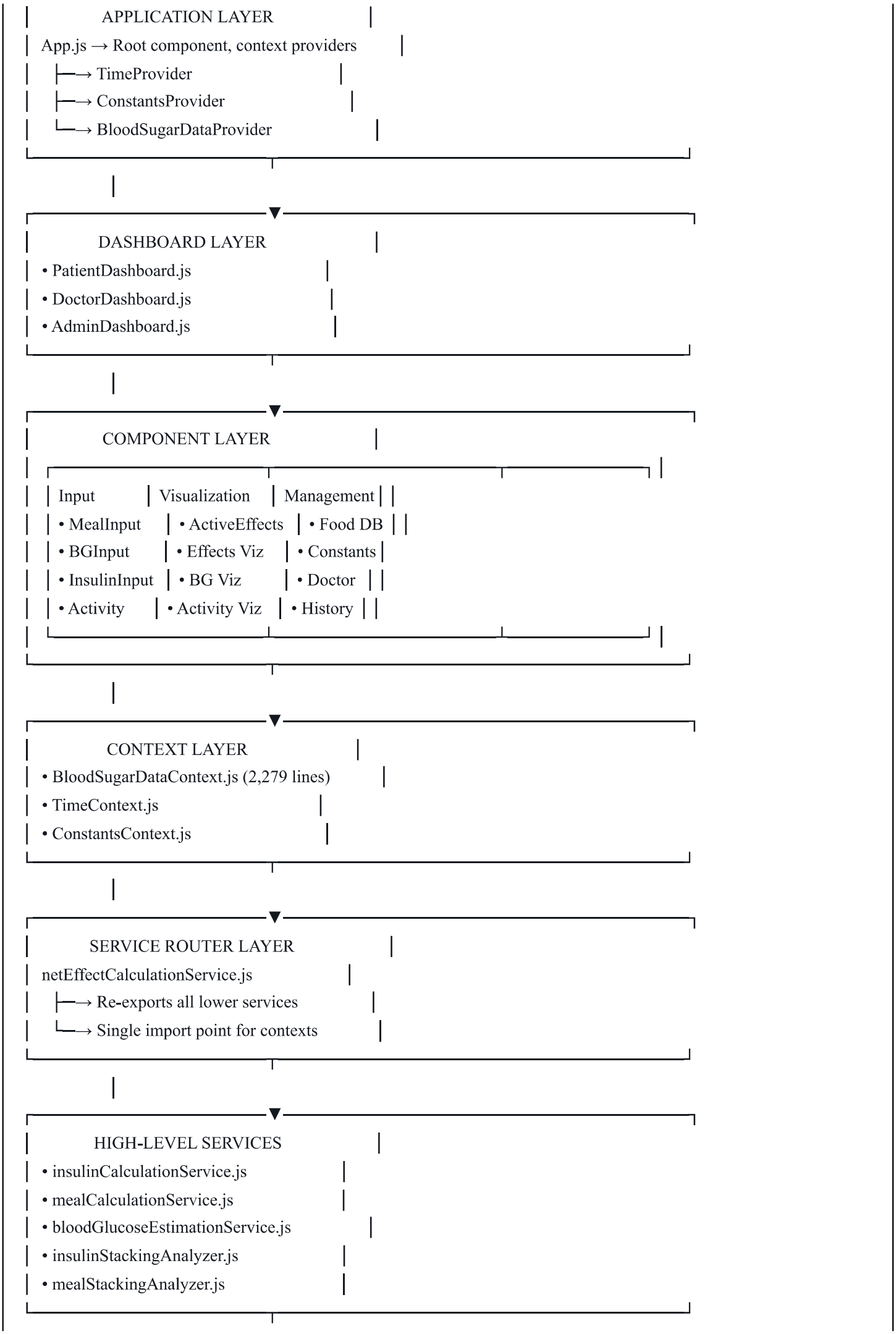
MongoDB Database: native_new

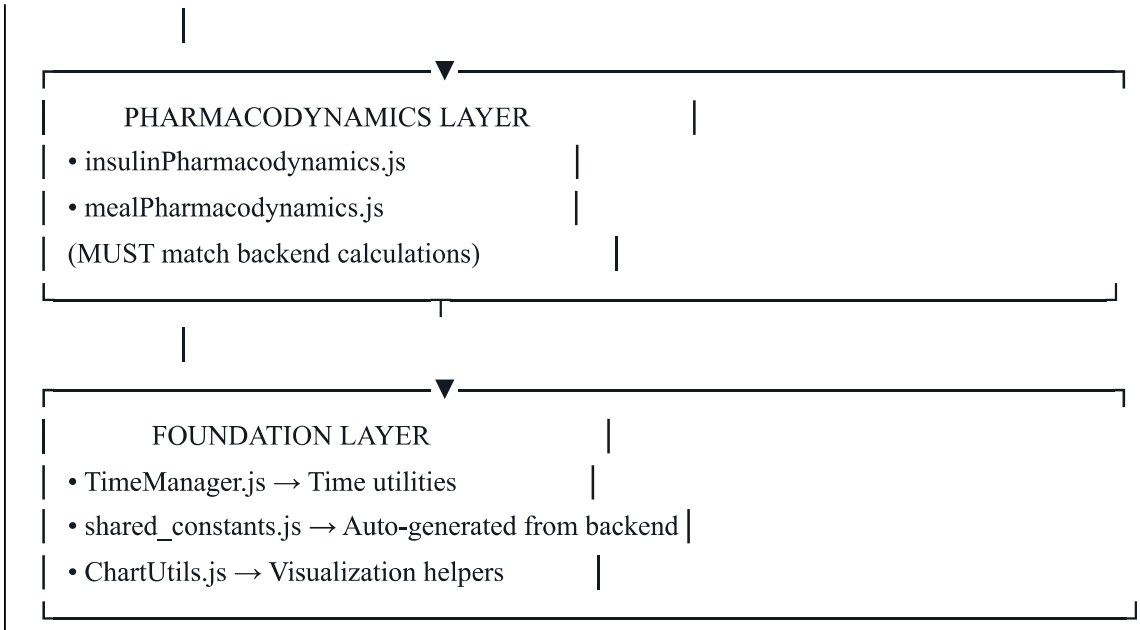
Collections:

Collection	Purpose
users	Patient/doctor/admin accounts - patient_constants (embedded) - assigned_doctors/patients
meals	Meal records with nutrition - foods array - total_nutrition - blood_sugar_id (link) - activity_ids (links)
blood_sugar	Glucose readings with baseline - baseline_calculation - meal_id (link)
activities	Physical activity logs - type (expected/completed) - level, impact, duration
medication_logs	Insulin dose tracking - medication, dose, unit - taken_at timestamp
medication_schedules	Recurring medication times - times array - active status
custom_foods	User-specific foods - per-user additions
favorite_foods	User favorites - quick access foods

FRONTEND STRUCTURE

Layer Architecture


APPLICATION LAYER
App.js → Root component, context providers
├→ TimeProvider
├→ ConstantsProvider
└→ BloodSugarDataProvider
DASHBOARD LAYER
• PatientDashboard.js
• DoctorDashboard.js
• AdminDashboard.js
COMPONENT LAYER
Input
• MealInput
• BGInput
• InsulinInput
• Activity
Visualization
• ActiveEffects
• Effects Viz
• BG Viz
• Activity Viz
Management
• Food DB
• Constants
• Doctor
• History
CONTEXT LAYER
• BloodSugarDataContext.js (2,279 lines)
• TimeContext.js
• ConstantsContext.js
SERVICE ROUTER LAYER
netEffectCalculationService.js
├→ Re-exports all lower services
└→ Single import point for contexts
HIGH-LEVEL SERVICES
• insulinCalculationService.js
• mealCalculationService.js
• bloodGlucoseEstimationService.js
• insulinStackingAnalyzer.js
• mealStackingAnalyzer.js



Component Organization

```
components/
├── Dashboards (3 files)
│   ├── PatientDashboard.js
│   ├── DoctorDashboard.js
│   └── AdminDashboard.js
│
├── Input Components (5 files)
│   ├── MealInput.js
│   ├── BloodSugarInput.js
│   ├── InsulinInput.js
│   ├── ActivityRecording.js
│   └── TimeInput.js
│
├── Visualization Components (8 files)
│   ├── ActiveEffectsDisplay.js (1,958 lines)
│   ├── EffectsVisualization.js
│   ├── BloodSugarVisualization.js
│   ├── ActivityVisualization.js
│   ├── GlycemicResponseTracker.js
│   ├── ClinicalValidationDashboard.js
│   ├── CGMMonitoringDashboard.js
│   └── SMBGMonitoringDashboard.js
│
├── Management Components (6 files)
│   ├── FoodDatabase.js
│   ├── FoodSection.js
│   ├── EnhancedPatientConstantsUI.js
│   ├── EnhancedPatientConstantsCalc.js
│   ├── MealHistory.js
```

```
│   └── DoctorManagement.js
│
├── Supporting Components (6 files)
│   ├── PatientConstants.js
│   ├── EnhancedPatientConstants.js
│   ├── MedicationSchedule.js
│   ├── NetEffectDisplay.js
│   ├── DoctorMealHistory.js
│   └── ImportExport.js
│
├── Charts (7 files)
│   ├── BloodSugarChart.js
│   ├── CombinedGlucoseInsulinChart.js
│   ├── InsulinVisualization.js
│   └── MealVisualizationmealsonly.js
│
└── Authentication (3 files)
    ├── SignIn.js
    ├── Register.js
    └── Test.js
```

Service Layer Organization

```
utils/
├── Foundation Services (2 files)
│   ├── TimeManager.js → Time precision utilities
│   └── ChartUtils.js → Visualization helpers
│
├── Pharmacodynamics Services (2 files)
│   ├── insulinPharmacodynamics.js → Insulin activity
│   └── mealPharmacodynamics.js → Meal absorption
│
├── Calculation Services (3 files)
│   ├── insulinCalculationService.js → IOB tracking
│   ├── mealCalculationService.js → MOB tracking
│   └── bloodGlucoseEstimationService.js → BG estimation
│
├── Analysis Services (2 files)
│   ├── insulinStackingAnalyzer.js → Stacking risk
│   └── mealStackingAnalyzer.js → Meal overlap
│
├── Integration Services (1 file)
│   └── netEffectCalculationService.js → Service router
│
├── Specialized Services (2 files)
│   ├── SMBGAnalysisService.js → SMBG optimization
```

```
│   └── validationMetrics.js → Clinical validation
│
├── Visualization Services (1 file)
│   └── confidenceVisualization.js → Confidence display
│
└── Data Services (1 file)
    └── DataImportHelper.js → Import functionality
```

📱 MOBILE STRUCTURE

React Native (Expo)

```
.expo/
├── devices.json      # Device configurations
└── README.md         # Expo documentation

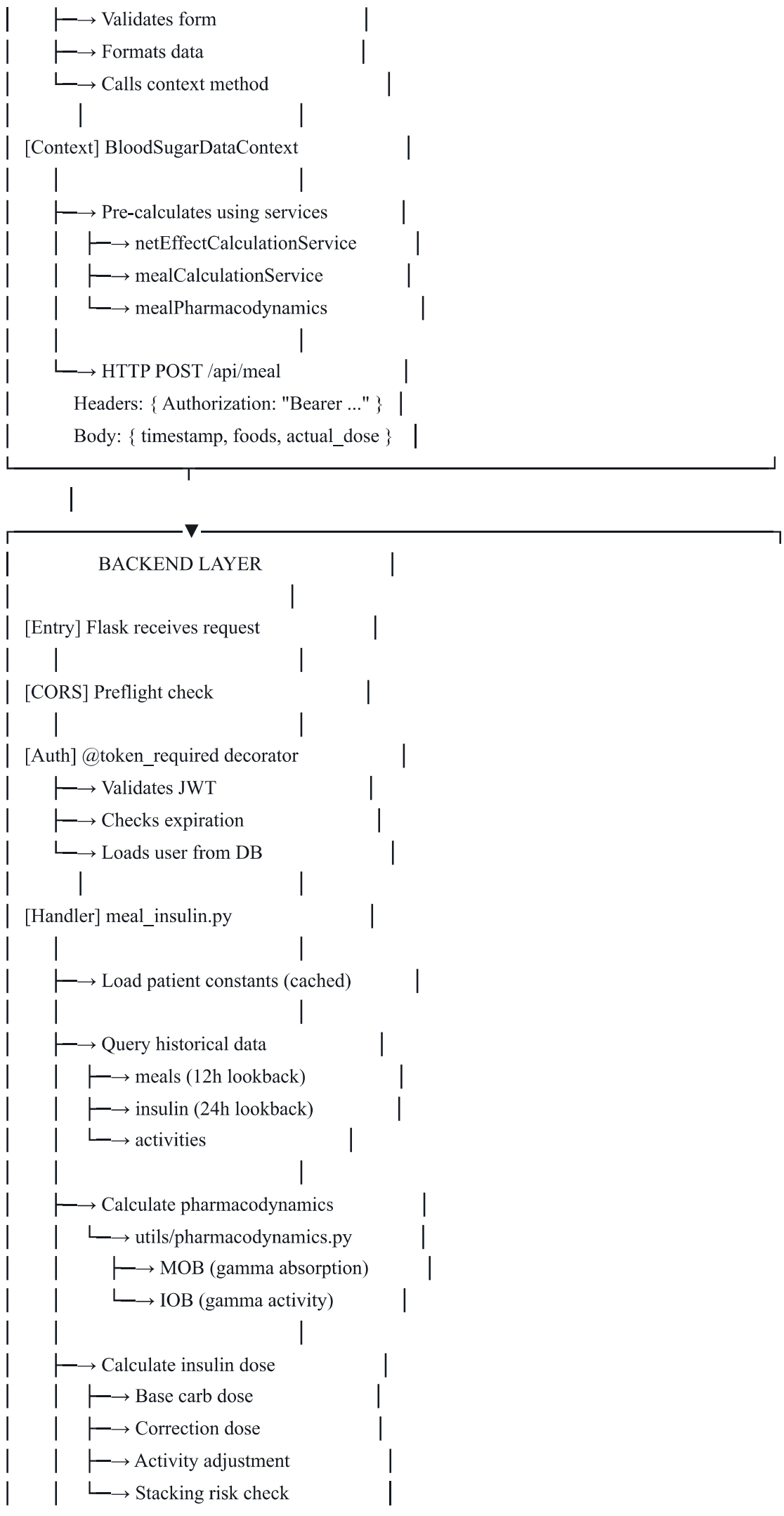
Future mobile app structure (planned):
mobile/
├── App.js            # Entry point
├── screens/          # Screen components
│   ├── Dashboard.js
│   ├── MealInput.js
│   └── History.js
├── components/       # Reusable components
├── navigation/       # Navigation setup
└── utils/            # Shared utilities with web
```

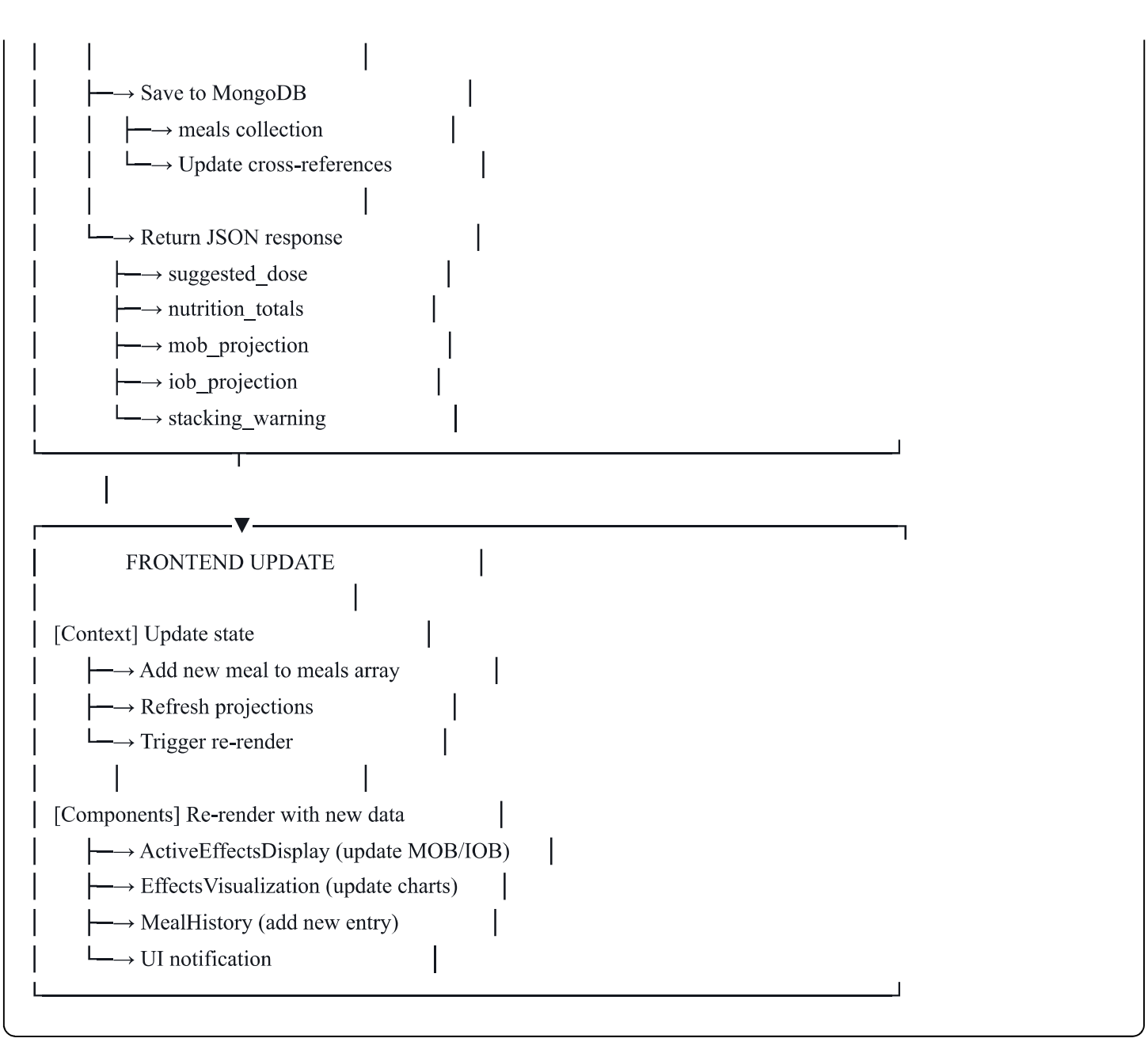
Note: Mobile app currently in development, sharing backend API with web app.

🔄 DATA FLOW ARCHITECTURE

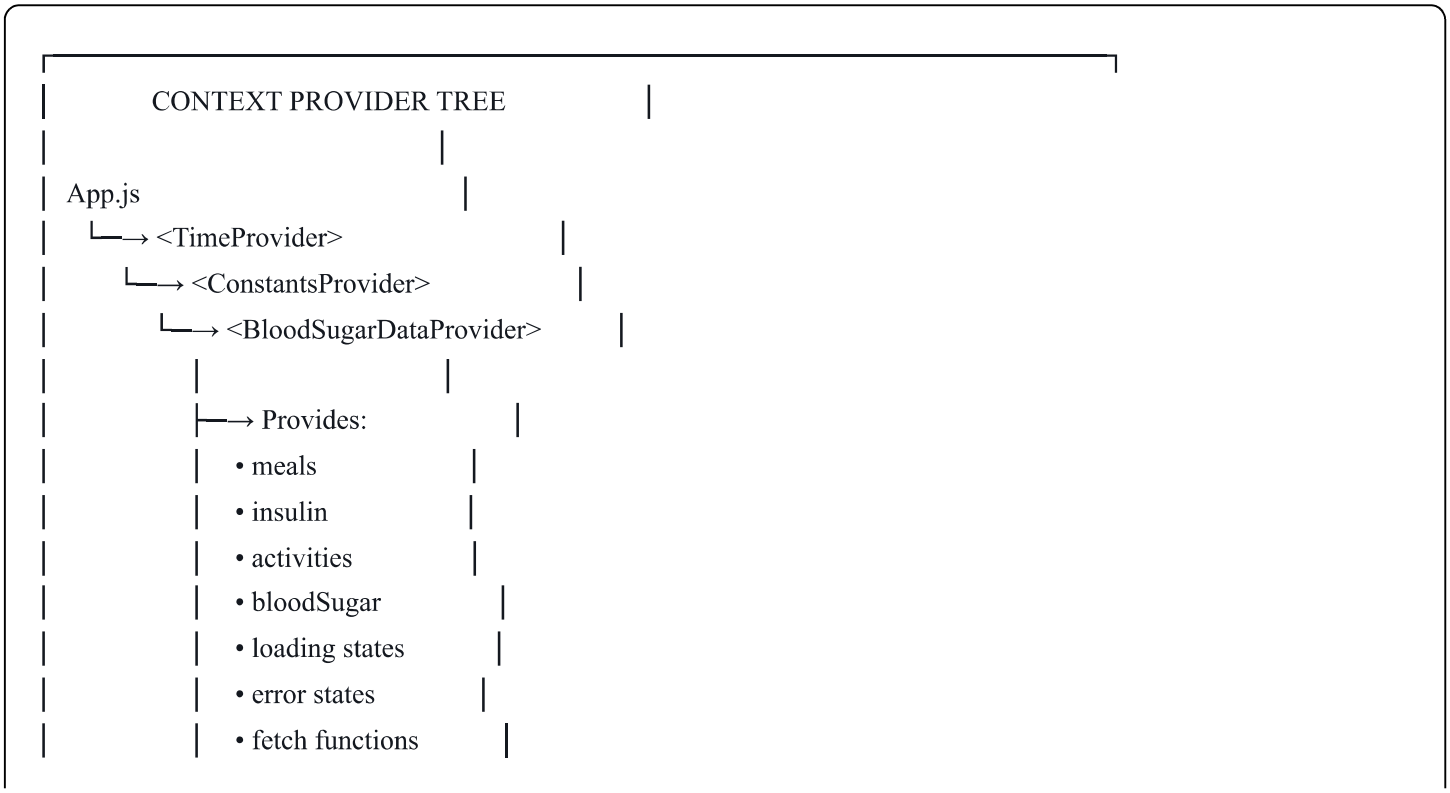
Complete Request Flow

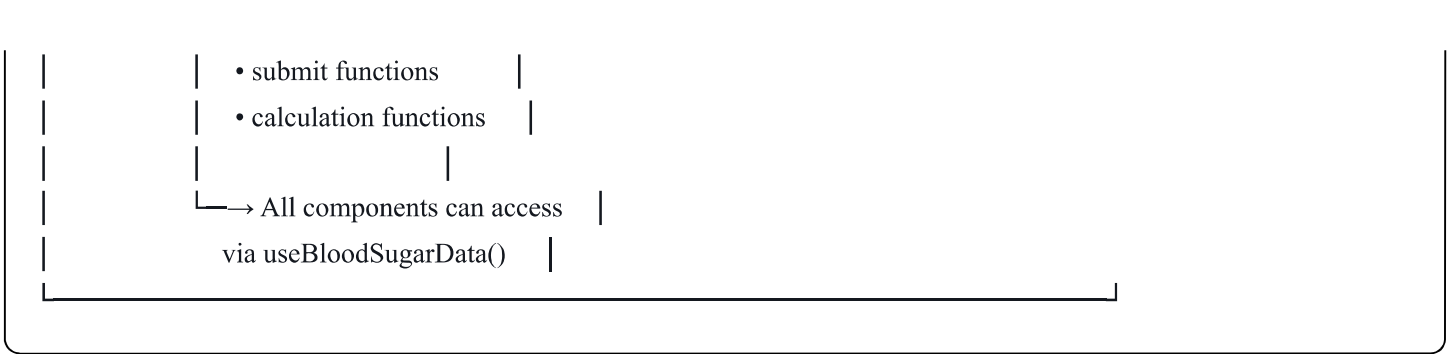
```
┌──────────────────────────────────────────────────────────┐
│                    USER ACTION                           │
│  "Submit Meal"                                           │
└──────────────────────┬───────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────┐
│                  FRONTEND LAYER                          │
│                                                          │
│  [Component] MealInput.js                                │
│       │                                                  │
```


├──→ Validates form
├──→ Formats data
└──→ Calls context method
[Context] BloodSugarDataContext
├──→ Pre-calculates using services
├──→ netEffectCalculationService
├──→ mealCalculationService
└──→ mealPharmacodynamics
└──→ HTTP POST /api/meal
Headers: { Authorization: "Bearer ..." }
Body: { timestamp, foods, actual_dose }
BACKEND LAYER
[Entry] Flask receives request
[CORS] Preflight check
[Auth] @token_required decorator
├──→ Validates JWT
├──→ Checks expiration
└──→ Loads user from DB
[Handler] meal_insulin.py
├──→ Load patient constants (cached)
├──→ Query historical data
├──→ meals (12h lookback)
├──→ insulin (24h lookback)
└──→ activities
├──→ Calculate pharmacodynamics
└──→ utils/pharmacodynamics.py
├──→ MOB (gamma absorption)
└──→ IOB (gamma activity)
├──→ Calculate insulin dose
├──→ Base carb dose
├──→ Correction dose
├──→ Activity adjustment
└──→ Stacking risk check



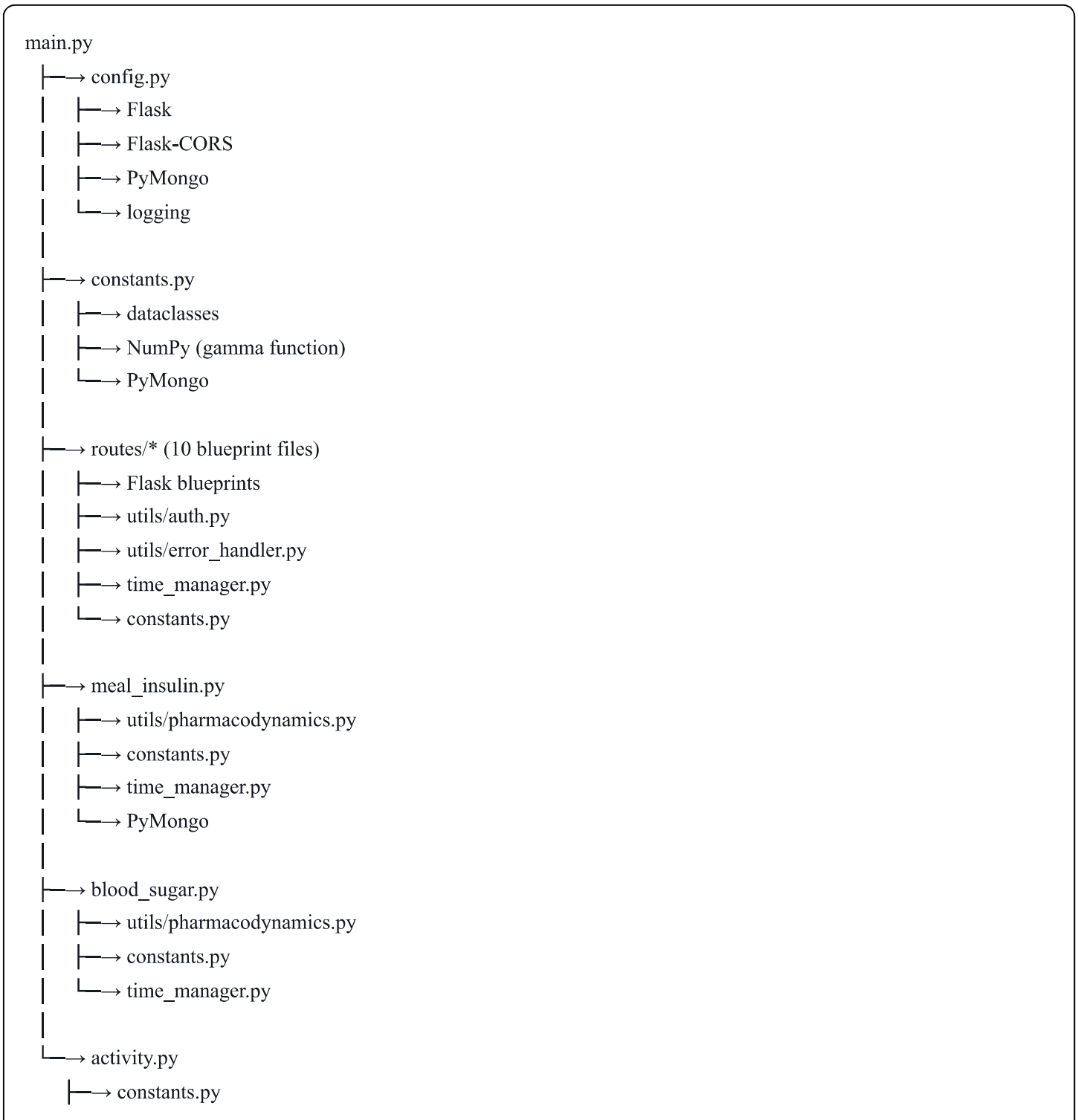
State Management Flow





🗺️ DEPENDENCY MAP

Backend Dependencies



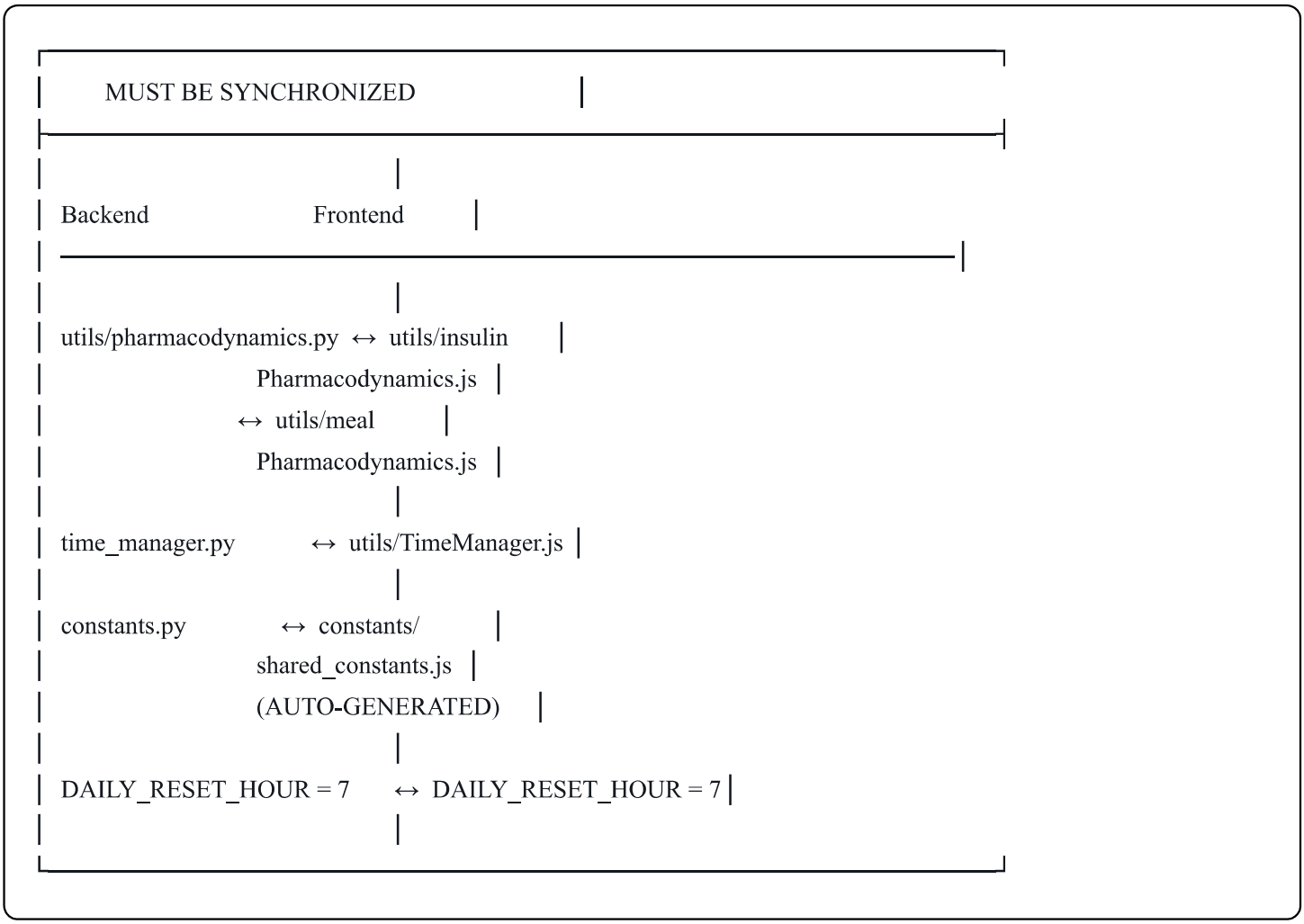
```
    ├──→ time_manager.py
    └──→ PyMongo
```

Frontend Dependencies

```
App.js
  ├──→ contexts/TimeContext.js
  │    └──→ utils/TimeManager.js
  │
  ├──→ contexts/ConstantsContext.js
  │    ├──→ constants/shared_constants.js (AUTO-GENERATED)
  │    └──→ axios
  │
  └──→ contexts/BloodSugarDataContext.js
       │
       ├──→ utils/netEffectCalculationService.js (SERVICE ROUTER)
       │    ├──→ utils/insulinCalculationService.js
       │    │    ├──→ utils/insulinPharmacodynamics.js
       │    │    │    └──→ utils/TimeManager.js
       │    │    └──→ utils/insulinStackingAnalyzer.js
       │    │         └──→ utils/insulinPharmacodynamics.js
       │    │
       │    ├──→ utils/mealCalculationService.js
       │    │    ├──→ utils/mealPharmacodynamics.js
       │    │    │    └──→ constants/shared_constants.js
       │    │    └──→ utils/mealStackingAnalyzer.js
       │    │         └──→ utils/mealPharmacodynamics.js
       │    │
       │    └──→ utils/bloodGlucoseEstimationService.js
       │         ├──→ utils/insulinPharmacodynamics.js
       │         ├──→ utils/mealPharmacodynamics.js
       │         └──→ utils/TimeManager.js
       │
       ├──→ axios (HTTP client)
       ├──→ TimeContext
       └──→ ConstantsContext

Components
  └──→ All import from contexts via hooks:
       ├──→ useBloodSugarData()
       ├──→ useTime()
       └──→ useConstants()
```

Critical Synchronization Points



🌐 API STRUCTURE

Endpoint Organization

```
Authentication
├─ POST   /api/signin
├─ POST   /api/register
├─ POST   /api/logout
├─ GET    /api/user
└─ PUT    /api/user

Patient Management
├─ GET    /api/patients
├─ GET    /api/patient/<id>
├─ PUT    /api/patient/<id>/constants
└─ GET    /api/patient/<id>/statistics

Doctor Portal
├─ GET    /api/doctors
├─ POST   /api/assign-patient
```

```
├── DELETE /api/unassign-patient
└── GET    /api/doctor/patients

Meal Operations
├── POST   /api/meal                # Submit meal + calculate insulin
├── GET    /api/meals               # Get meal history
├── GET    /api/meals-only          # MOB tracking endpoint
├── POST   /api/meal/calculate      # Preview (no save)
├── DELETE /api/meal/<id>
├── POST   /api/import-meals
└── POST   /api/repair-imported-meals

Blood Glucose
├── POST   /api/blood-sugar         # Submit reading + baseline calc
├── GET    /api/blood-sugar         # Get history
└── GET    /api/blood-sugar/<patient_id> # Doctor view

Medication
├── POST   /api/medication          # Log insulin dose
├── GET    /api/medication          # Get medication history
├── POST   /api/medication-schedule    # Create schedule
├── GET    /api/medication-schedules   # Get schedules
├── PUT    /api/medication-schedule/<id>
└── DELETE /api/medication-schedule/<id>

Activity
├── POST   /api/record-activities   # Log activity
├── GET    /api/activity-history    # Get history
├── GET    /api/activities          # Paginated list
└── GET    /api/activity/<id>

Food Database
├── GET    /api/foods               # Search foods
├── POST   /api/custom-food         # Add custom food
├── GET    /api/custom-foods        # Get user foods
├── PUT    /api/custom-food/<id>
├── DELETE /api/custom-food/<id>
├── POST   /api/favorite-food       # Add favorite
└── GET    /api/favorite-foods

Import/Export
├── POST   /api/import              # Import CSV/JSON
└── GET    /api/export              # Export data

Testing (development only)
├── GET    /api/test
└── POST   /api/test-calculation
```

```
API Versioning
└─ /api/v1/* → Proxied to /api/*        # Mobile compatibility
```

Request/Response Patterns

Authentication Required:

```
Headers: {
  "Content-Type": "application/json",
  "Authorization": "Bearer <jwt_token>"
}
```

Typical Response Structure:

```javascript
// Success
{
  "success": true,
  "data": { ... },
  "message": "Operation successful"
}

// Error
{
  "error": "Error message",
  "details": { ... }  // Optional
}

// Validation Error
{
  "error": "Validation failed",
  "errors": {
    "field1": "Error message",
    "field2": "Error message"
  }
}
```

🧩 COMPONENT HIERARCHY

Patient Dashboard

```
PatientDashboard
```

```
├──→ TimeInput
├──→ MealInput
│    ├──→ FoodSection (multiple)
│    │    └──→ FoodDatabase
│    └──→ InsulinInput (optional embed)
│
├──→ BloodSugarInput
├──→ ActivityRecording
├──→ ActiveEffectsDisplay
│    ├──→ IOB Section
│    ├──→ MOB Section
│    └──→ Net Effect Section
│
├──→ EffectsVisualization
│    └──→ Recharts components
│
├──→ BloodSugarVisualization
│    └──→ Recharts components
│
├──→ GlycemicResponseTracker
├──→ MealHistory
├──→ PatientConstants
└──→ ImportExport
```

Doctor Dashboard

```
DoctorDashboard
├──→ DoctorManagement
│    └──→ Patient assignment UI
│
├──→ Patient selector
│
├──→ DoctorMealHistory
│    └──→ Patient meal view
│
├──→ ClinicalValidationDashboard
│    └──→ Validation metrics
│
├──→ CGMMonitoringDashboard
│    └──→ Continuous monitoring
│
└──→ EnhancedPatientConstantsUI
     └──→ Tuning interface
```

Admin Dashboard

```
AdminDashboard
├──→ User management
│    ├──→ Create users
│    ├──→ Edit users
│    └──→ Delete users
│
├──→ System statistics
├──→ Food database management
└──→ System configuration
```

📖 FILE REFERENCE GUIDE

Quick File Lookup

Backend Critical Files

File	Lines	Purpose	Priority
`main.py`	~200	Entry point, blueprint registration	CRITICAL
`config.py`	~150	CORS, MongoDB, logging	CRITICAL
`constants.py`	~800	Patient constants, pharma profiles	CRITICAL
`utils/pharmacodynamics.py`	~500	MOB/IOB calculations	CRITICAL
`meal_insulin.py`	~600	Meal processing, dose calculation	CRITICAL
`time_manager.py`	~300	Time precision utilities	HIGH
`blood_sugar.py`	~400	Blood glucose, baseline	HIGH
`activity.py`	~300	Activity tracking	MEDIUM
`routes/auth_routes.py`	~200	Authentication	HIGH
`routes/meal_routes.py`	~300	Meal history, MOB	MEDIUM

Frontend Critical Files

File	Lines	Purpose	Priority
`BloodSugarDataContext.js`	2,279	Central state hub	CRITICAL
`netEffectCalculationService.js`	1,695	Service router	CRITICAL
`bloodGlucoseEstimationService.js`	1,681	BG estimation	CRITICAL
`ActiveEffectsDisplay.js`	1,958	Main effects display	HIGH
`ChartUtils.js`	1,377	Visualization helpers	MEDIUM
`insulinPharmacodynamics.js`	988	Insulin calculations	CRITICAL
`mealPharmacodynamics.js`	679	Meal calculations	CRITICAL
`TimeManager.js`	~400	Time utilities	CRITICAL
`shared_constants.js`	812	Auto-generated constants	CRITICAL
`App.js`	~150	Root component	HIGH

File Modification Guidelines

⚠️ **NEVER EDIT MANUALLY**:

- `frontend/src/constants/shared_constants.js` (auto-generated)

⚠️ **EDIT WITH EXTREME CAUTION**:

- `backend/utils/pharmacodynamics.py` (medical calculations)
- `frontend/utils/*Pharmacodynamics.js` (must match backend)
- `backend/time_manager.py` (time precision)
- `frontend/utils/TimeManager.js` (must match backend)
- `backend/constants.py` (affects entire system)

✅ **SAFE TO EDIT**:

- UI components (visual changes)
- CSS files (styling)
- Route handlers (business logic)
- New feature files

🔄 **REQUIRES SYNC**:

- Any pharmacodynamics changes (backend ↔ frontend)

- Time management changes (backend ↔ frontend)
- Constant changes (backend → frontend export)

🎓 FOR DEVELOPERS

Starting a New Feature

1. Identify Layer:

```
UI Change → Component layer
Calculation → Service/Utils layer
API Endpoint → Backend routes layer
Database → Models/Collections
```

2. Follow Pattern:

```
Backend: Blueprint → Route → Business Logic → Utils
Frontend: Component → Context → Service → Pharmacodynamics
```

3. Maintain Sync:

```
If touching pharmacodynamics → Update both backend & frontend
If adding constants → Add to backend, re-export
If changing time logic → Update both managers
```

Debugging Strategy

1. Identify Layer:

```
UI issue → Component/CSS
State issue → Context
Calculation issue → Service/Utils
API issue → Backend route
Data issue → Database/Model
```

2. Check Logs:

```
Backend: Flask logs (extensive)
Frontend: Console (DEBUG flags)
Database: MongoDB logs
```

3. Trace Flow:

Component → Context → Service → API → Backend → Database
Follow the chain to find break point

Adding New Endpoints

Backend:

```python
# 1. Create route in appropriate blueprint
@meal_bp.route('/api/new-endpoint', methods=['POST'])
@token_required
@api_error_handler
def new_endpoint():
    # Implementation

# 2. Register blueprint in main.py (if new)
# 3. Test with Postman/curl
```

Frontend:

```javascript
// 1. Add function to context
const newFunction = async () => {
  const response = await axios.post('/api/new-endpoint', data);
  // Update state
};

// 2. Provide in context value
// 3. Use in component via hook
```

📝 SUMMARY

Project Statistics

Backend:

- **Language**: Python 3.8+
- **Framework**: Flask 2.x
- **Database**: MongoDB (native_new)
- **Total Files**: ~30
- **Lines of Code**: ~10,000-15,000

Frontend:

- **Language**: JavaScript (ES6+)
- **Framework**: React 18.x
- **State**: Context API
- **Total Files**: ~60
- **Lines of Code**: ~15,000-20,000

Total Application:

- **Combined LOC**: ~25,000-35,000
- **API Endpoints**: ~40+
- **MongoDB Collections**: 8
- **React Components**: 33

Architecture Summary

Pattern: Full-stack monorepo with:

- Backend: Blueprint-based Flask API
- Frontend: Service layer routing with React Context
- Mobile: React Native (Expo) - in development
- Database: Document-based MongoDB
- Sync: Auto-generated constants, mirrored calculations

Key Principles:

1. Medical-grade accuracy (German S3 Guidelines)
2. Time precision (millisecond storage, second calculations)
3. Backend-frontend sync (pharmacodynamics)
4. Centralized state (React Context)
5. Service layer routing (clear dependencies)

Document Version: 3.0
Last Updated: February 11, 2026
Maintained By: DiaTwin Development Team